

高矮个子排队

题目描述

现在有一队 **小朋友**，他们高矮不同，我们以正整数数组表示这一队小朋友的身高，如数组{5,3,1,2,3}。

我们现在希望小朋友排队，以“高”“矮”“高”“矮”顺序排列，每一个“高”位置的小朋友要比相邻的位置高或者相等；每一个“矮”位置的小朋友要比相邻的位置矮或者相等；

要求小朋友们移动的距离和最小，第一个从“高”位开始排，输出最小 **移动距离** 即可。

例如，在示范小队{5,3,1,2,3}中，{5, 1, 3, 2, 3}是排序结果。

{5, 2, 3, 1, 3} 虽然也满足“高”“矮”“高”“矮”顺序排列，但小朋友们的移动距离大，所以不是最优结果。

移动距离的定义如下所示：

第二位小朋友移到第三位小朋友后面，移动距离为1，若移动到第四位小朋友后面，移动距离为2；

输入描述

排序前的小朋友，以英文空格的正整数：

4 3 5 7 8

注：小朋友<100个

输出描述

排序后的小朋友，以英文空格分割的正整数：4 3 7 5 8

备注：4（高）3（矮）7（高）5（矮）8（高），输出结果为最小移动距离，只有5和7交换了位置，移动距离都是1。

用例

输入	4 1 3 5 2
输出	4 1 5 2 3
说明	无

输入	1 1 1 1 1
输出	1 1 1 1 1
说明	相邻位置可以相等

输入	xxx
输出	[]
说明	出现非法参数情况， 返回空数组。

题目解析

感觉本题的用例1是存在问题的

4 1 3 5 2

的最小移动距离应该是1，即让5和2交换位置，变为：4 1 3 2 5，这样的话，也满足：高 矮 高 矮 高。

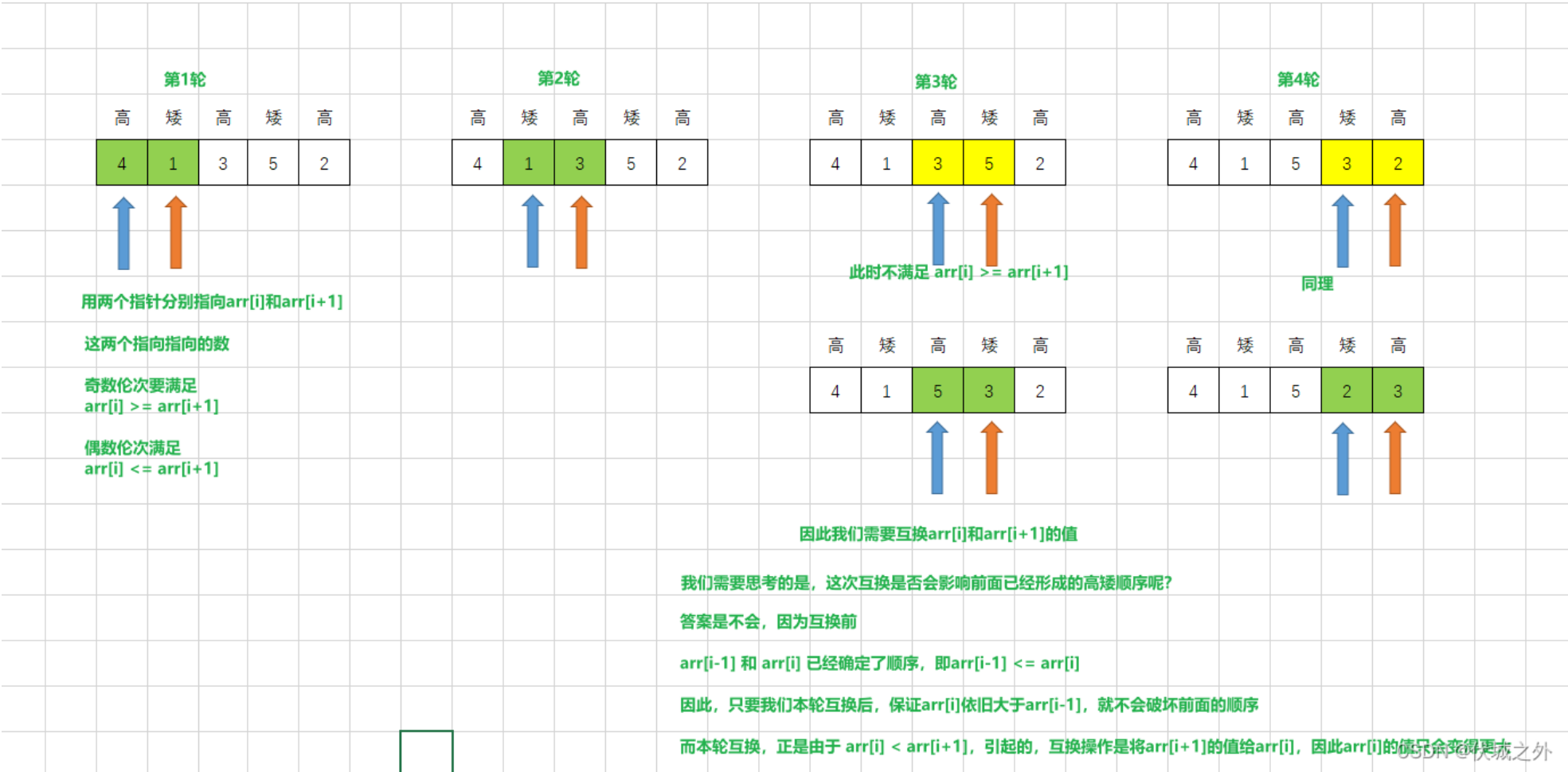
而用例输出的逻辑是先让3和5交换，变为 4 1 5 3 2，再让3和2交换，变为 4 1 5 2 3，这样的话也满足：高 矮 高 矮 高，但是却交换了两次，也就是说移动距离是2。

难道说，用例逻辑是，必须要从第一个小朋友开始，每当遇到不符合要求的排队顺序，就必须交换位置吗？

那么题目描述中又为何要强调最小距离呢？

感觉这题有点自相矛盾。

我这里按照符合用例要求的逻辑写的代码，示意图如下



但是这种算法是不满足最小距离要求的。

JS算法源码

```
1 | const rl = require("readline").createInterface({ input: process.stdin });
2 | var iter = rl[Symbol.asyncIterator]()
3 | const readline = async () => (await iter.next()).value;
```

运行

```

4 |
5 | void (async function () {
6 |   const nums = (await readline()).split(" ").map(Number);
7 |
8 |   // 出现非法参数情况， 返回空数组
9 |   for(let num of nums) {
10 |     if(isNaN(num)) {
11 |       console.log("[]");
12 |       return;
13 |     }
14 |   }
15 |
16 |   for (let i = 0; i < nums.length - 1; i++) {
17 |     // 偶数轮次，高矮，不符合则需要交换
18 |     // 奇数轮次，矮高，不符合则需要交换
19 |     let needSwap = i % 2 == 0 ? nums[i] < nums[i + 1] : nums[i] > nums[i + 1];
20 |
21 |     if (needSwap) {
22 |       let tmp = nums[i];
23 |       nums[i] = nums[i + 1];
24 |       nums[i + 1] = tmp;
25 |     }
26 |   }
27 |
28 |   console.log(nums.join(" "));
29 | })();

```

Java算法源码

```

1 | import java.util.Arrays;
2 | import java.util.Scanner;
3 | import java.util.StringJoiner;
4 |
5 | public class Main {

```

```

6 // 输入获取 7 | public static void main(String[] args) {
8     Scanner sc = new Scanner(System.in);
9
10    try {
11        int[] nums = Arrays.stream(sc.nextLine().split(" ")).mapToInt(Integer::parseInt).toArray();
12
13        for (int i = 0; i < nums.length - 1; i++) {
14            boolean needSwap = i % 2 == 0 ? nums[i] < nums[i + 1] : nums[i] > nums[i + 1];
15
16            if (needSwap) {
17                int tmp = nums[i];
18                nums[i] = nums[i + 1];
19                nums[i + 1] = tmp;
20            }
21        }
22
23        StringJoiner sj = new StringJoiner(" ");
24        for (int num : nums) {
25            sj.add(num + "");
26        }
27
28        System.out.println(sj);
29    } catch (Exception e) {
30        System.out.println("[]");
31    }
32 }
33 }

```

Python算法源码

```

1 try:
2     nums = list(map(int, input().split()))
3
4     for i in range(len(nums) - 1):

```

```
5         needSwap = nums[i] < nums[i + 1] if i % 2 == 0 else nums[i] > nums[i + 1] 6 |
7         if needSwap:
8             nums[i], nums[i + 1] = nums[i + 1], nums[i]
9
10        print(" ".join(map(str, nums)))
11    except ValueError:
12        print("[]")
```

C算法源码

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4
5  #define MAX_SIZE 100
6
7  int main() {
8      int nums[MAX_SIZE];
9      int nums_size = 0;
10
11      char token[100];
12      while (scanf("%s", token) != EOF) {
13          int i = 0;
14          while (token[i] >= '0' && token[i] <= '9') {
15              i++;
16          }
17
18          if (token[i] != '\0') {
19              puts("[]");
20              return 0;
21          }
22
23          nums[nums_size++] = atoi(token);
24      }
25
```

运行

```

26 |     for (int i = 0; i < nums_size - 1; i++) {
27 |         bool needSwap = i % 2 == 0 ? nums[i] < nums[i + 1] : nums[i] > nums[i + 1];
28 |
29 |         if (needSwap) {
30 |             int tmp = nums[i];
31 |             nums[i] = nums[i + 1];
32 |             nums[i + 1] = tmp;
33 |         }
34 |     }
35 |
36 |     for (int i = 0; i < nums_size; i++) {
37 |         printf("%d ", nums[i]);
38 |     }
39 |
40 |     return 0;
41 | }

```

C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | int main() {
6 |     vector<int> nums;
7 |
8 |     string s;
9 |     while (cin >> s) {
10 |         for (const auto &c: s) {
11 |             if (!isdigit(c)) {
12 |                 cout << "[" << endl;
13 |                 return 0;
14 |             }
15 |         }

```

```
16 |
17 |         nums.emplace_back(stoi(s));
18 |     }
19 |
20 |     for (int i = 0; i < nums.size() - 1; i++) {
21 |         bool needSwap = i % 2 == 0 ? nums[i] < nums[i + 1] : nums[i] > nums[i + 1];
22 |
23 |         if (needSwap) {
24 |             int tmp = nums[i];
25 |             nums[i] = nums[i + 1];
26 |             nums[i + 1] = tmp;
27 |         }
28 |     }
29 |
30 |     for (int num: nums) {
31 |         cout << num << " ";
32 |     }
33 |
34 |     return 0;
35 | }
```